Векторное представление слов

&&

Харитон Пицик

2026-04-29

Клуб машинного обучения

В общем смысле, текст – это точно такой же признак, как и любая другая ранее рассмотренная сущность.

Текст как признак

&&

В общем смысле, текст – это точно такой же признак, как и любая другая ранее рассмотренная сущность.

Следовательно, нам точно также нужно научиться её как-то кодировать.

Вспомним, как мы ранее обрабатывали признаки:

- Числа – в некоторых случаях стандартизация / нормализация
- Категориальные признаки – One Hot Encoding, Label Encoding etc.
- Картинки – CNN

Открытым вопросом остаётся обработка текста, так как здесь сразу большое количество подводных камней:

Открытым вопросом остаётся обработка текста, так как здесь сразу большое количество подводных камней:

- Предложения могут быть разной длины. Как с этим справляться?

Текст как признак

&&

Открытым вопросом остаётся обработка текста, так как здесь сразу большое количество подводных камней:

- Предложения могут быть разной длины. Как с этим справляться?
- Для каждого возможного слова должно существовать численное представление

Открытым вопросом остаётся обработка текста, так как здесь сразу большое количество подводных камней:

- Предложения могут быть разной длины. Как с этим справляться?
- Для каждого возможного слова должно существовать численное представление
- Как учитывать синонимы?

Открытым вопросом остаётся обработка текста, так как здесь сразу большое количество подводных камней:

- Предложения могут быть разной длины. Как с этим справляться?
- Для каждого возможного слова должно существовать численное представление
- Как учитывать синонимы?
- Как учитывать контекст, *семантику*?

Постановка задачи

&&

Формально, мы хотим построить отображение $f : V \to \mathbb{R}^d$.

V – vocabulary, размер словаря; d – размерность вектора.

Формально, мы хотим построить отображение $f : V \to \mathbb{R}^d$.

V – vocabulary, размер словаря; d – размерность вектора.

Получается, что для каждого слова мы должны получить вектор, который наиболее оптимален для решения поставленной задачи. Этот вектор и называют *эмбеддингом*.

Поскольку это векторное пространство, то хотелось бы интерпретировать расстояние между этими векторами как степень похожести слов (чем 2 вектора ближе друг к другу, тем более похожи исходные слова)

Обычно это делается с помощью косинусного расстояния, поскольку оно учитывает не только длину векторов, но и их направление:

$$\cos(a, b) = \frac{a * b}{\|a\| \ \|b\|}$$

Хорошее представление

Хорошее представление

&&

Давайте для начала подумаем о том, какое текстовое представление мы хотим по итогу получить:

Хорошее представление

&&

Давайте для начала подумаем о том, какое текстовое представление мы хотим по итогу получить:

- Интерпретируемое сходство слов;

Хорошее представление

&&

Давайте для начала подумаем о том, какое текстовое представление мы хотим по итогу получить:

- Интерпретируемое сходство слов;
- Сходство документов;

Хорошее представление &&

Давайте для начала подумаем о том, какое текстовое представление мы хотим по итогу получить:

- Интерпретируемое сходство слов;
- Сходство документов;
- Синтаксические и семантические свойства;

Хорошее представление

&&

Давайте для начала подумаем о том, какое текстовое представление мы хотим по итогу получить:

- Интерпретируемое сходство слов;
- Сходство документов;
- Синтаксические и семантические свойства;
- Контекст;

Хорошее представление

&&

Давайте для начала подумаем о том, какое текстовое представление мы хотим по итогу получить:

- Интерпретируемое сходство слов;
- Сходство документов;
- Синтаксические и семантические свойства;
- Контекст;
- Обобщение на новые слова.

Первые попытки.

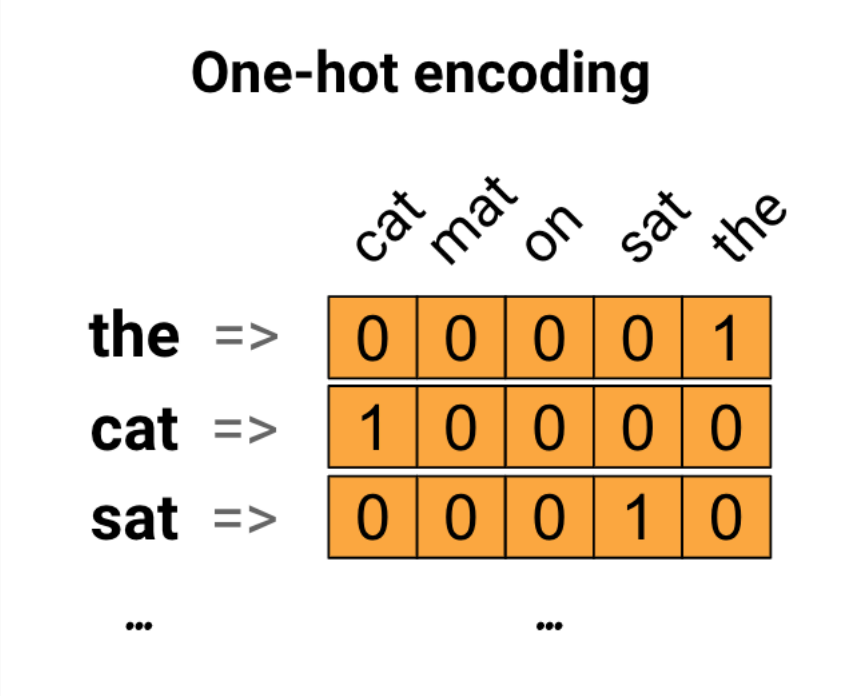
Первые попытки. &&

Самым тривиальным методом может оказаться простая нумерация слов в предложении (или букв в слове), однако мы тем самым создаём несуществующую структуру, поэтому этот вариант сразу отпадает.

Первые попытки. &&

Самым тривиальным методом может оказаться простая нумерация слов в предложении (или букв в слове), однако мы тем самым создаём несуществующую структуру, поэтому этот вариант сразу отпадает.

Другое дело, что можно использовать уже знакомый One Hot Encoding для этих же целей: создадим вектор длины V, и будем в векторе ставить единицу на i-й позиции, если слово в словаре на i-й позиции встречается в тексте, иначе 0.


One-hot encoding
cat
mat
on
sat
the
the => 0 0 0 0 1
cat => 1 0 0 0 0
sat => 0 0 0 1 0
...
...

Первые попытки. &&

Однако такой подход имеет ряд недостатков:

- Векторы становятся очень разреженными, из-за чего с ними неудобно работать;
- Мы не имеем ни малейшего понятия о контексте и семантике;
- Все слова одинаково далеки друг от друга;
- Нет обобщения.

Переход от текстов к документам. &&

Рассмотрим немного иной подход: пусть у нас есть 2 документа:

Документ 1: “кошка сидит на окне”

Документ 2: “собака сидит у двери”

Переход от текстов к документам. &&

Рассмотрим немного иной подход: пусть у нас есть 2 документа:

Документ 1: “кошка сидит на окне”

Документ 2: “собака сидит у двери”

Построим словарь: [кошка, собака, сидит, на, у, окне, двери]. А дальше просто построим вектор частот:

Переход от текстов к документам. &&

Рассмотрим немного иной подход: пусть у нас есть 2 документа:

Документ 1: “кошка сидит на окне”

Документ 2: “собака сидит у двери”

Построим словарь: [кошка, собака, сидит, на, у, окне, двери]. А дальше просто построим вектор частот:

Документ 1: [1, 0, 1, 1, 0, 1, 0]

Документ 2: [0, 1, 1, 0, 1, 0, 1]

Этот метод называется Bag of Words (BoW).

Что на самом деле делает BoW?

&&

Каждый документ превращается в вектор из пространства $\mathbb{R}^{|V|}$, где каждая координата отвечает слову из словаря.

Что на самом деле делает BoW?

&&

Каждый документ превращается в вектор из пространства $\mathbb{R}^{|V|}$, где каждая координата отвечает слову из словаря.

Если собрать такие векторы для всех текстов, то получится *матрица «документ–терм»*:

- по строкам — документы,
- по столбцам — слова,
- в ячейке — сколько раз слово встретилось в документе.

Что на самом деле делает BoW? &&

Каждый документ превращается в вектор из пространства $\mathbb{R}^{|V|}$, где каждая координата отвечает слову из словаря.

Если собрать такие векторы для всех текстов, то получится *матрица «документ–терм»*:

- по строкам — документы,
- по столбцам — слова,
- в ячейке — сколько раз слово встретилось в документе.

Это уже позволяет:

- сравнивать документы между собой;
- искать ключевые слова в тексте;
- обучать простые линейные модели классификации.

Что на самом деле делает BoW?

&&

В названии метода скрывается его главный недостаток: он никак не учитывает порядок слов, что является достаточно сильной проблемой: “кот съел мышь” и “мышь съела кота” в таком случае могут быть близкими по смыслу предложениями.

Что на самом деле делает BoW? &&

В названии метода скрывается его главный недостаток: он никак не учитывает порядок слов, что является достаточно сильной проблемой: “кот съел мышь” и “мышь съела кота” в таком случае могут быть близкими по смыслу предложениями.

Кроме того, у BoW есть и другая проблема: слова вроде “и”, “в”, “на” встречаются почти везде, но почти не помогают отличать один документ от другого.

Что на самом деле делает BoW? &&

В названии метода скрывается его главный недостаток: он никак не учитывает порядок слов, что является достаточно сильной проблемой: “кот съел мышь” и “мышь съела кота” в таком случае могут быть близкими по смыслу предложениями.

Кроме того, у BoW есть и другая проблема: слова вроде “и”, “в”, “на” встречаются почти везде, но почти не помогают отличать один документ от другого.

Значит, одних частот мало: хочется взвешивать слова по их *информативности*.

В попытках построить крутые эмбеддинги... &&

Можно задать достаточно разумный вопрос: учитывая, что слова – дискретны, возможно, есть какая-то возможность определить похожесть на основе контекста?

В попытках построить крутые эмбеддинги... &&

Можно задать достаточно разумный вопрос: учитывая, что слова – дискретны, возможно, есть какая-то возможность определить похожесть на основе контекста?

Эти рассуждения приходят к центральной гипотезе всех эмбеддингов – *гипотезе о распределении.*

В классическом виде она восходит к работам Зеллига Харриса (1954), а популярную формулировку дал Джон Фёрт (1957): ““You shall know a word by the company it keeps.””

Гипотеза о распределении

&&

В классическом виде она восходит к работам Зеллига Харриса (1954), а популярную формулировку дал Джон Фёрт (1957): ““You shall know a word by the company it keeps.””

Идея простая: если два слова встречаются в похожих окружениях, то и значение у них, вероятно, похоже.

Гипотеза о распределении

&&

В классическом виде она восходит к работам Зеллига Харриса (1954), а популярную формулировку дал Джон Фёрт (1957): ““You shall know a word by the company it keeps.””

Идея простая: если два слова встречаются в похожих окружениях, то и значение у них, вероятно, похоже.

Например, слова “кошка” и “собака” могут встречаться рядом с глаголами “бежит”, “спит”, “ест”, а слова “окно” и “дверь” – рядом с “открыть”, “закрыть”, “войти”.

Слабая форма гипотезы: распределение по документам

&&

Контекст можно понимать по-разному.

Слабая форма гипотезы: распределение по документам &&

Контекст можно понимать по-разному.

- *Локально*: какие слова стоят рядом в окне из нескольких токенов;

Слабая форма гипотезы: распределение по документам

&&

Контекст можно понимать по-разному.

- *Локально*: какие слова стоят рядом в окне из нескольких токенов;
- *Глобально*: в каких документах слово вообще встречается и насколько часто.

Слабая форма гипотезы: распределение по документам

&&

Контекст можно понимать по-разному.

- *Локально*: какие слова стоят рядом в окне из нескольких токенов;
- *Глобально*: в каких документах слово вообще встречается и насколько часто.

BoW использует именно вторую, более грубую идею: слово характеризуется своим распределением по коллекции документов.

Слабая форма гипотезы: распределение по документам &&

Контекст можно понимать по-разному.

- *Локально*: какие слова стоят рядом в окне из нескольких токенов;
- *Глобально*: в каких документах слово вообще встречается и насколько часто.

BoW использует именно вторую, более грубую идею: слово характеризуется своим распределением по коллекции документов.

Тогда похожие документы должны иметь похожие наборы важных слов, а важность слова можно оценивать по тому, *насколько оно характерно для данного документа и насколько редко встречается в остальных.*

Слабая форма гипотезы: распределение по документам

&&

Если слово часто встречается в конкретном документе, то оно, скорее всего, важно для его темы. Но если это же слово встречается почти *во всех* документах корпуса, то отличать документы оно уже не помогает.

Слабая форма гипотезы: распределение по документам

Если слово часто встречается в конкретном документе, то оно, скорее всего, важно для его темы. Но если это же слово встречается почти *во всех* документах корпуса, то отличать документы оно уже не помогает.

Отсюда появляется естественная идея разложить вес слова на две части:

Слабая форма гипотезы: распределение по документам &&

Если слово часто встречается в конкретном документе, то оно, скорее всего, важно для его темы. Но если это же слово встречается почти *во всех* документах корпуса, то отличать документы оно уже не помогает.

Отсюда появляется естественная идея разложить вес слова на две части:

- *TF* (term frequency) — насколько часто слово встретилось в документе;

Слабая форма гипотезы: распределение по документам

&&

Если слово часто встречается в конкретном документе, то оно, скорее всего, важно для его темы. Но если это же слово встречается почти *во всех* документах корпуса, то отличать документы оно уже не помогает.

Отсюда появляется естественная идея разложить вес слова на две части:

- *TF* (term frequency) — насколько часто слово встретилось в документе;
- *IDF* (inverse document frequency) — насколько это слово редкое во всём корпусе.

Слабая форма гипотезы: распределение по документам &&

Если слово часто встречается в конкретном документе, то оно, скорее всего, важно для его темы. Но если это же слово встречается почти *во всех* документах корпуса, то отличать документы оно уже не помогает.

Отсюда появляется естественная идея разложить вес слова на две части:

- *TF* (term frequency) — насколько часто слово встретилось в документе;
- *IDF* (inverse document frequency) — насколько это слово редкое во всём корпусе.

Итоговый вес – произведение этих двух величин.

Для слова t в документе d обычно пишут:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

Для слова t в документе d обычно пишут:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

Одна из стандартных формул для обратной документной частоты:

$$\text{idf}(t) = \log\left(\frac{N}{\text{df}(t)}\right)$$

Для слова t в документе d обычно пишут:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

Одна из стандартных формул для обратной документной частоты:

$$\text{idf}(t) = \log\left(\frac{N}{\text{df}(t)}\right)$$

где N – число документов в корпусе, а $\text{df}(t)$ – в скольких документах встретилось слово t.

Для слова t в документе d обычно пишут:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

Одна из стандартных формул для обратной документной частоты:

$$\text{idf}(t) = \log\left(\frac{N}{\text{df}(t)}\right)$$

где N – число документов в корпусе, а $\text{df}(t)$ – в скольких документах встретилось слово t.

Интерпретация:

- высокий TF: слово важно именно в этом тексте;
- высокий IDF: слово редкое и потому различающее;
- высокий TF-IDF: хорошее ключевое слово для документа.

Интуиция на примере

&&

Слово “нейросеть” может встретиться 12 раз в конспекте про глубокое обучение и лишь в 2 документах из 1000. Тогда его вес будет большим.

Интуиция на примере

&&

Слово “нейросеть” может встретиться 12 раз в конспекте про глубокое обучение и лишь в 2 документах из 1000. Тогда его вес будет большим.

А слово “это” может встретиться 20 раз, но почти во всех документах корпуса. Несмотря на высокий TF, его IDF будет маленьким, и итоговый вес окажется низким.

Интуиция на примере

&&

Слово “нейросеть” может встретиться 12 раз в конспекте про глубокое обучение и лишь в 2 документах из 1000. Тогда его вес будет большим.

А слово “это” может встретиться 20 раз, но почти во всех документах корпуса. Несмотря на высокий TF, его IDF будет маленьким, и итоговый вес окажется низким.

Именно поэтому TF-IDF обычно работает заметно лучше простого подсчёта частот.

Недостатки TF-IDF

&&

TF-IDF уже использует идею распределения, но делает это в очень грубой форме:

Недостатки TF-IDF

&&

TF-IDF уже использует идею распределения, но делает это в очень грубой форме:

- не учитывает порядок слов;

Недостатки TF-IDF

&&

TF-IDF уже использует идею распределения, но делает это в очень грубой форме:

- не учитывает порядок слов;
- не различает значения многозначных слов;

Недостатки TF-IDF

&&

TF-IDF уже использует идею распределения, но делает это в очень грубой форме:

- не учитывает порядок слов;
- не различает значения многозначных слов;
- не умеет напрямую кодировать семантическую близость вроде “кот” “кошка”.

Word2vec

От частот к предсказанию контекста

&&

TF-IDF и Bag of Words используют *глобальную статистику* по корпусу, но почти не учитывают локальный контекст слова.

Word2vec предлагает другой взгляд:

От частот к предсказанию контекста

&&

TF-IDF и Bag of Words используют *глобальную статистику* по корпусу, но почти не учитывают локальный контекст слова.

Word2vec предлагает другой взгляд:

- слово должно иметь такой вектор, чтобы по нему можно было *предсказывать соседние слова*

От частот к предсказанию контекста

&&

TF-IDF и Bag of Words используют *глобальную статистику* по корпусу, но почти не учитывают локальный контекст слова.

Word2vec предлагает другой взгляд:

- слово должно иметь такой вектор, чтобы по нему можно было *предсказывать соседние слова*
- по соседним словам можно было предсказать само слово.

От частот к предсказанию контекста

&&

TF-IDF и Bag of Words используют *глобальную статистику* по корпусу, но почти не учитывают локальный контекст слова.

Word2vec предлагает другой взгляд:

- слово должно иметь такой вектор, чтобы по нему можно было *предсказывать соседние слова*
- по соседним словам можно было предсказать само слово.

Иными словами, эмбеддинг учится не из “таблицы частот”, а из задачи предсказания контекста.

Общая идея Word2vec

&&

Пусть весь корпус текстов представлен как длинная последовательность слов:

$$w_1, w_2, w_3, ..., w_T$$

Общая идея Word2vec

&&

Пусть весь корпус текстов представлен как длинная последовательность слов:

$$w_1, w_2, w_3, ..., w_T$$

Дальше по этой последовательности движется окно ширины K:

- выбирается центральное слово;
- рассматриваются слова слева и справа от него;
- модель учится по одним словам предсказывать другие.

Общая идея Word2vec

&&

Пусть весь корпус текстов представлен как длинная последовательность слов:

$$w_1, w_2, w_3, \ldots, w_T$$

Дальше по этой последовательности движется окно ширины K:

- выбирается центральное слово;
- рассматриваются слова слева и справа от него;
- модель учится по одним словам предсказывать другие.

Это и есть реализация гипотезы о распределении: *слова, встречающиеся в похожих контекстах, должны получать похожие векторы.*

Два вектора для каждого слова

&&

В классическом Word2vec каждое слово имеет два представления:

- *входной эмбеддинг* — когда слово подаётся модели на вход;
- *выходной эмбеддинг* — когда слово нужно предсказать.

Два вектора для каждого слова

&&

В классическом Word2vec каждое слово имеет два представления:

- *входной эмбеддинг* — когда слово подаётся модели на вход;
- *выходной эмбеддинг* — когда слово нужно предсказать.

Обычно их обозначают как:

$$u_w \in \mathbb{R}^d, \quad v_w \in \mathbb{R}^d$$

Два вектора для каждого слова

&&

В классическом Word2vec каждое слово имеет два представления:

- *входной эмбеддинг* — когда слово подаётся модели на вход;
- *выходной эмбеддинг* — когда слово нужно предсказать.

Обычно их обозначают как:

$$u_w \in \mathbb{R}^d, \quad v_w \in \mathbb{R}^d$$

На практике после обучения чаще всего используют один из этих векторов (или их комбинацию) как итоговый эмбеддинг слова.

Skip-gram &&

В модели *Skip-gram* известно центральное слово, и по нему нужно предсказать слова в его окрестности.

В модели *Skip-gram* известно центральное слово, и по нему нужно предсказать слова в его окрестности.

Например, для фразы ““кошка сидит на окне”” если центральное слово – “сидит”, то модель должна уметь предсказывать слова “кошка”, “на”, а при более широком окне – и “окне”.

Skip-gram &&

В модели *Skip-gram* известно центральное слово, и по нему нужно предсказать слова в его окрестности.

Например, для фразы ““кошка сидит на окне”” если центральное слово – “сидит”, то модель должна уметь предсказывать слова “кошка”, “на”, а при более широком окне – и “окне”.

То есть задача Skip-gram: *по слову восстановить его контекст.*

Почему это работает?

Если два слова встречаются в похожих контекстах, то для хорошего предсказания их векторы должны стать похожими.

Почему это работает? &&

Если два слова встречаются в похожих контекстах, то для хорошего предсказания их векторы должны стать похожими.

Например, слова “кошка” и “собака” могут часто встречаться рядом со словами:

- “бежит”,
- “спит”,
- “ест”,
- “домашний”.

Почему это работает?

&&

Если два слова встречаются в похожих контекстах, то для хорошего предсказания их векторы должны стать похожими.

Например, слова “кошка” и “собака” могут часто встречаться рядом со словами:

- “бежит”,
- “спит”,
- “ест”,
- “домашний”.

Тогда модель будет подстраивать их эмбеддинги так, чтобы они хорошо предсказывали сходные наборы контекстных слов.

Skip-gram: вероятностная идея

&&

Вероятность того, что слово w_o встретится в контексте слова w_c, задаётся через сходство их векторов.

Skip-gram: вероятностная идея

&&

Вероятность того, что слово w_o встретится в контексте слова w_c, задаётся через сходство их векторов.

Чем больше скалярное произведение $u_{w_c}^T v_{w_o}$, тем более совместимыми считаются эти слова.

Skip-gram: вероятностная идея

&&

Вероятность того, что слово w_o встретится в контексте слова w_c, задаётся через сходство их векторов.

Чем больше скалярное произведение $u_{w_c}^T v_{w_o}$, тем более совместимыми считаются эти слова.

После этого применяется SoftMax, чтобы получить вероятности по всему словарю.

Обучение Skip-gram

&&

Модель проходит по корпусу и максимизирует вероятность правильных контекстных слов для каждого центрального слова. В результате слова с похожими окружениями получают близкие эмбеддинги.

В модели *CBOW* всё наоборот: по соседним словам нужно предсказать *центральное* слово.

В модели *CBOW* всё наоборот: по соседним словам нужно предсказать *центральное* слово.

Например, если известны слова ““кошка … на окне”” то модель должна предсказать, что в центре, вероятно, находится слово “сидит”.

В модели *CBOW* всё наоборот: по соседним словам нужно предсказать *центральное* слово.

Например, если известны слова ““кошка … на окне”” то модель должна предсказать, что в центре, вероятно, находится слово “сидит”.

То есть задача CBOW: *по контексту восстановить пропущенное слово.*

Как работает CBOW

&&

Сначала берутся эмбеддинги соседних слов. Затем они объединяются в один вектор контекста (в простейшем случае – суммированием или усреднением).

Как работает CBOW

&&

Сначала берутся эмбеддинги соседних слов. Затем они объединяются в один вектор контекста (в простейшем случае – суммированием или усреднением).

После этого модель сравнивает полученный вектор со всеми словами словаря и выбирает наиболее вероятное центральное слово.

Skip-gram vs CBOW

&&

Обе модели обучают эмбеддинги, но делают это по-разному:

Skip-gram vs CBOW

&&

Обе модели обучают эмбеддинги, но делают это по-разному:

- Skip-gram сильнее фокусируется на каждом отдельном слове;

Skip-gram vs CBOW

&&

Обе модели обучают эмбеддинги, но делают это по-разному:

- Skip-gram сильнее фокусируется на каждом отдельном слове;
- CBOW использует усреднённый контекст и часто работает быстрее.

Что получается после обучения? &&

После обучения каждое слово получает плотный вектор небольшой размерности.

В отличие от One Hot Encoding:

Что получается после обучения? &&

После обучения каждое слово получает плотный вектор небольшой размерности.

В отличие от One Hot Encoding:

- эти векторы *не разрежены*

Что получается после обучения? &&

После обучения каждое слово получает плотный вектор небольшой размерности.

В отличие от One Hot Encoding:

- эти векторы *не разрежены*
- слова уже не одинаково далеки друг от друга;

Что получается после обучения? &&

После обучения каждое слово получает плотный вектор небольшой размерности.

В отличие от One Hot Encoding:

- эти векторы *не разрежены*
- слова уже не одинаково далеки друг от друга;
- появляется геометрическая структура, связанная со смыслом.

Семантические регулярности

&&

Оказалось, что в пространстве Word2vec возникают интересные закономерности.

Семантические регулярности

&&

Оказалось, что в пространстве Word2vec возникают интересные закономерности.

Например, близкими оказываются слова:

- student teacher, school, university;
- car truck, vehicle;
- country nation, region.

Семантические регулярности

&&

Оказалось, что в пространстве Word2vec возникают интересные закономерности.

Например, близкими оказываются слова:

- student teacher, school, university;
- car truck, vehicle;
- country nation, region.

Более того, некоторые отношения оказываются почти линейными:

$$v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}$$

Семантические регулярности

&&

Оказалось, что в пространстве Word2vec возникают интересные закономерности.

Например, близкими оказываются слова:

- student teacher, school, university;
- car truck, vehicle;
- country nation, region.

Более того, некоторые отношения оказываются почти линейными:

$$v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}$$

Это означает, что эмбеддинги кодируют не только близость слов, но и некоторые типы отношений между ними.

Ограничения Word2vec

&&

Несмотря на успех, у Word2vec есть ограничения:

Ограничения Word2vec

&&

Несмотря на успех, у Word2vec есть ограничения:

- слово получает *один* вектор вне зависимости от контекста;

Ограничения Word2vec

&&

Несмотря на успех, у Word2vec есть ограничения:

- слово получает *один* вектор вне зависимости от контекста;
- многозначные слова не разделяются по значениям;

Ограничения Word2vec

&&

Несмотря на успех, у Word2vec есть ограничения:

- слово получает *один* вектор вне зависимости от контекста;
- многозначные слова не разделяются по значениям;
- модель плохо работает с совсем редкими и новыми словами.

Зачем идти дальше? &&

У Word2vec есть важное ограничение: эмбеддинг есть только у тех слов, которые модель видела при обучении;

Зачем идти дальше? &&

У Word2vec есть важное ограничение: эмбеддинг есть только у тех слов, которые модель видела при обучении;

Для языков с богатой морфологией это особенно заметно: слова “кот”, “кота”, “коту”, “котик” связаны между собой, но Word2vec рассматривает их как независимые токены.

В **fastText** слово представляется не только как целый токен, но и как набор символьных *n-грамм*.

Зачем идти дальше? &&

В **fastText** слово представляется не только как целый токен, но и как набор символьных *n-грамм*.

Например, слово “person” можно дополнить специальными символами границ: “[person]”

В **fastText** слово представляется не только как целый токен, но и как набор символьных *n-грамм*.

Например, слово “person” можно дополнить специальными символами границ: “[person]”

Тогда его 3-граммы будут такими:

- “[pe”;
- “per”;
- “ers”;
- “rso”;
- “son”;
- “on]”.

Как строится эмбеддинг слова? &&

Во входной части модели вектор слова заменяется на сумму векторов его символьных n-грамм (обычно рассматривают $3 \leq n \leq 6$), а также часто добавляют и само слово целиком.

То есть смысл слова извлекается не только из внешнего контекста, но и из его *внутренней структуры*.

Как строится эмбеддинг слова? &&

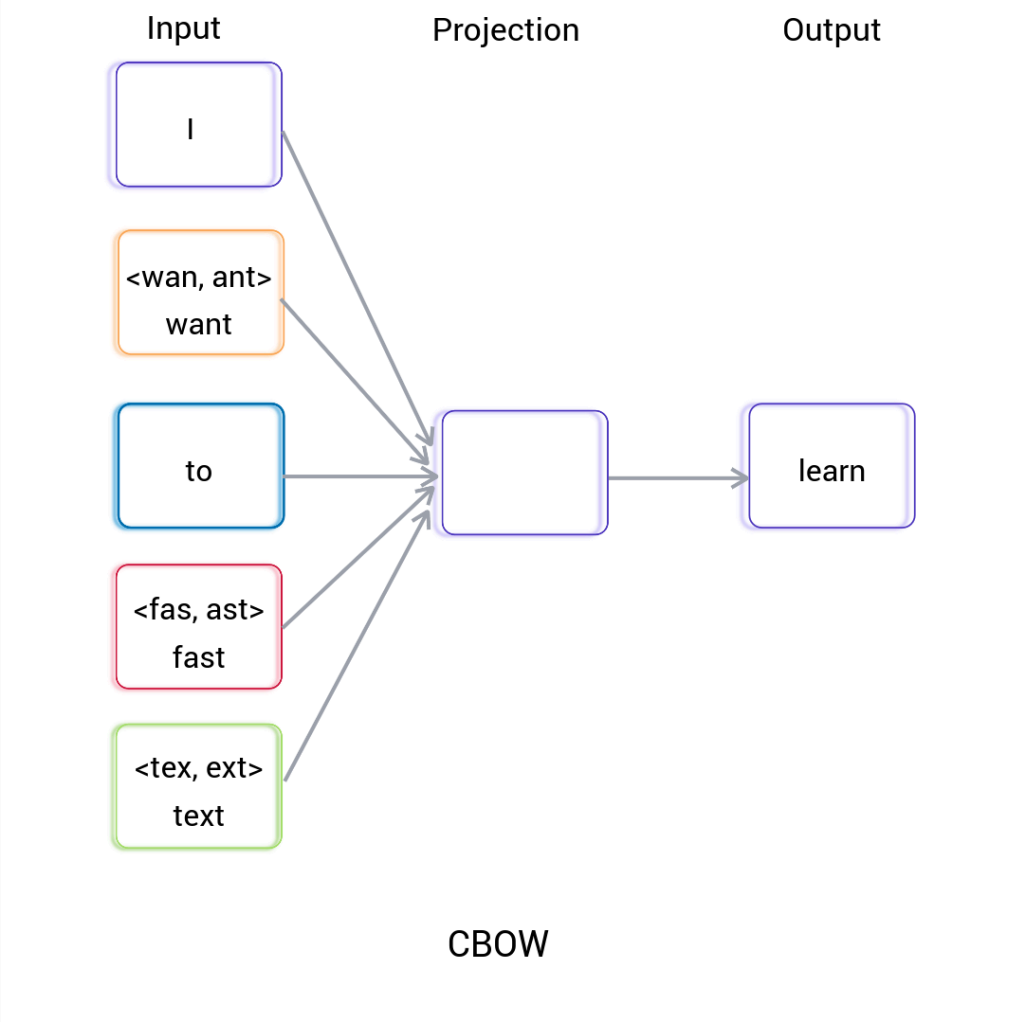
Во входной части модели вектор слова заменяется на сумму векторов его символьных n-грамм (обычно рассматривают $3 \leq n \leq 6$), а также часто добавляют и само слово целиком.

То есть смысл слова извлекается не только из внешнего контекста, но и из его *внутренней структуры*.

Это позволяет переносить информацию между похожими словоформами и родственными словами.

Как строится эмбеддинг слова?

&&



Почему это полезно?

Если модель видит новое слово или слово с опечаткой, она всё равно может построить его эмбеддинг по уже знакомым n-граммам.

Почему это полезно? &&

Если модель видит новое слово или слово с опечаткой, она всё равно может построить его эмбеддинг по уже знакомым n-граммам.

Например, слова

- run,
- running,
- runner

имеют общую часть, и fastText учитывает это напрямую, а не только через похожие контексты.

Почему это полезно? &&

Если модель видит новое слово или слово с опечаткой, она всё равно может построить его эмбеддинг по уже знакомым n-граммам.

Например, слова

- run,
- running,
- runner

имеют общую часть, и fastText учитывает это напрямую, а не только через похожие контексты.

Поэтому fastText обычно лучше Word2vec работает:

- с редкими словами;
- с морфологически богатыми языками;
- с частично незнакомыми словами.